Core helpers

Study guide 1: periods, readers, web, the report registry, context processors and apps.ready()

What this guide covers

File	Lines	Gives every report app
`core/periods.py`	14	month_range() and previous_month()
`core/readers.py`	18	read_csv() with clear errors
`core/web.py`	20	download() and sheet_preview() for views
`core/reports.py`	18	the Report registry
`core/context_processors.py`	6	the menu's data, in every template
`commission/apps.py, activity/apps.py`	12 each	how an app registers itself
`core/views.py, core/urls.py`	9, 6	the home page

How to use it

- Read Part 1 and the pictures once.
- Rebuild each file in Part 3 by typing it, file by file, running its check before moving on.
- Do the checkpoints in Part 5 from memory. All of them were run against the real project.

Part 1. Think before you write code

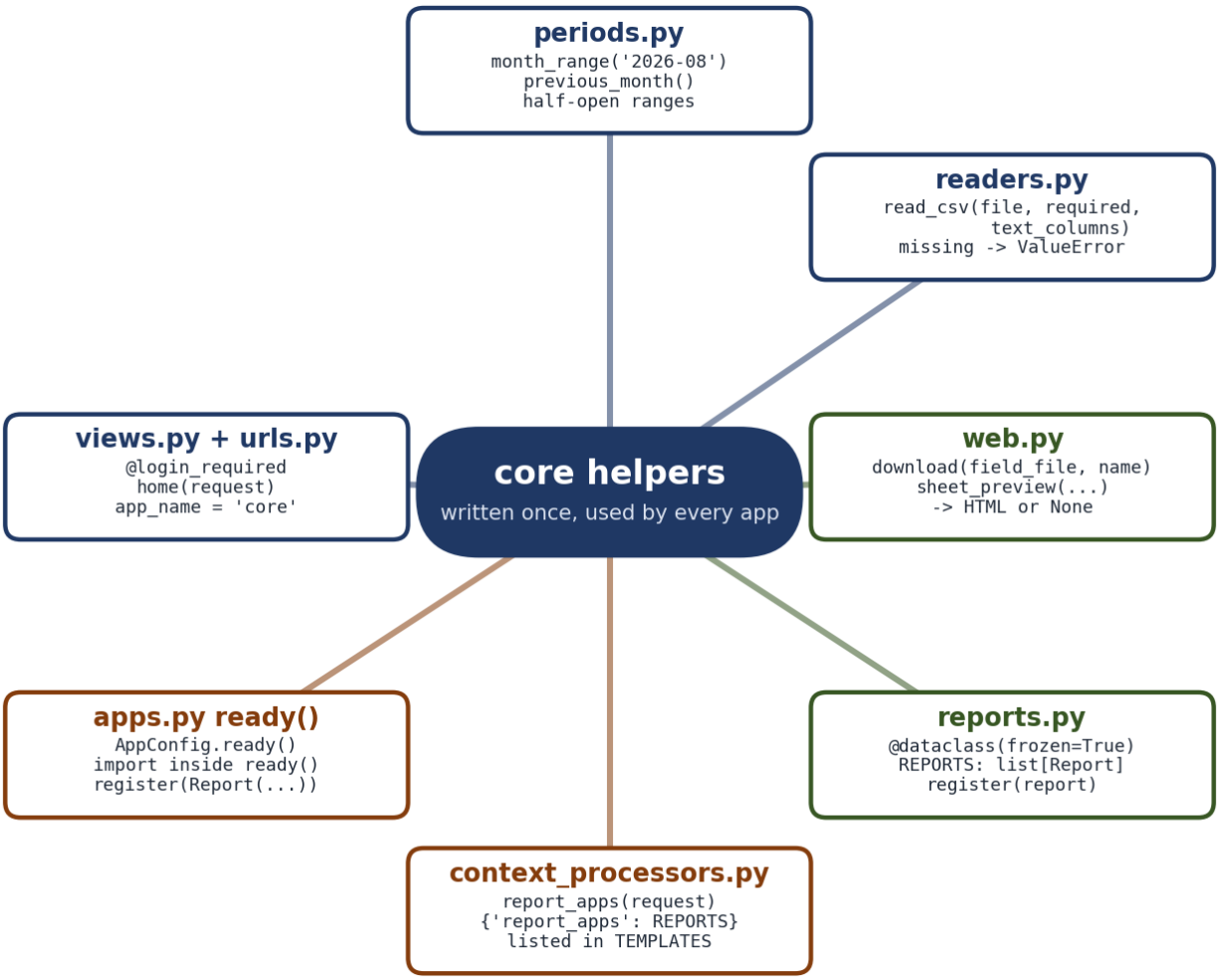
Each of these files is small, and that's the point. A helper earns its place in `core` when **a second app would otherwise copy it**. Before writing one, answer three questions:

1. **Who calls it?** A view, a services module, Django itself?
2. **Does it need Django?** If not, keep Django out, so services and notebooks can use it.
3. **What should happen when the input is wrong?** Return something harmless, or raise a clear error?

Helper	Called by	Needs Django?	On bad input
`month_range, previous_month`	views, services, notebooks	No	pandas raises (a bad month never reaches it: the form checks first)
`read_csv`	services.read_brokers / read_clients	No	raises ValueError with a message the user can act on
`download, sheet_preview`	views	Yes (FileResponse, FieldFile)	sheet_preview returns None
`Report, register`	each app's AppConfig.ready()	No	a repeated key is ignored
`report_apps`	Django, for every template	Yes (takes a request)	-

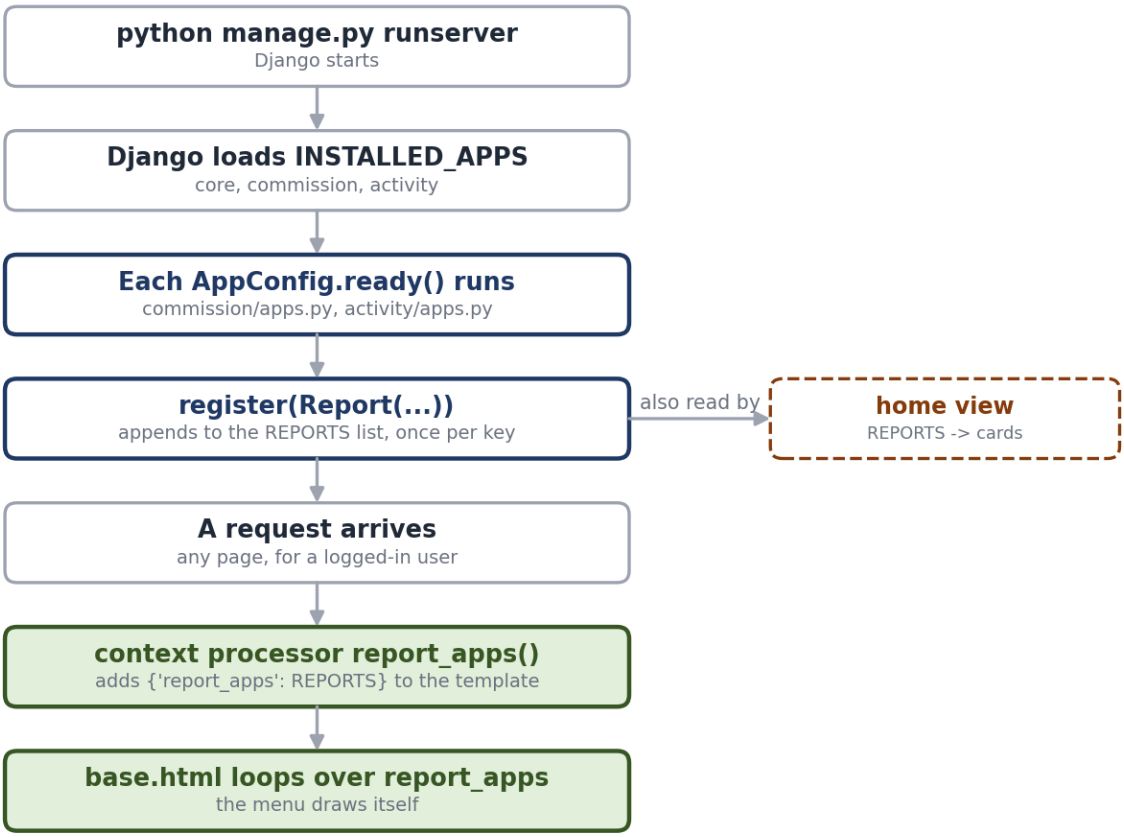
The rule that decides where code goes: **pure helpers** (no Django) can be imported anywhere, including a notebook; **web helpers** belong to views. Mixing them makes services.py depend on Django, and then you can't test or reuse it as plain pandas.

All the helpers as a mindmap


periods.py
month_range('2026-08')
previous_month()
half-open ranges
readers.py
read_csv(file, required,
text_columns)
missing -> ValueError
views.py + urls.py
@login_required
home(request)
app_name = 'core'
core helpers
written once, used by every app
web.py
download(field_file, name)
sheet_preview(...)
-> HTML or None
apps.py ready()
AppConfig.ready()
import inside ready()
register(Report(...))
reports.py
@dataclass(frozen=True)
REPORTS: list[Report]
register(report)
context_processors.py
report_apps(request)
{'report_apps': REPORTS}
listed in TEMPLATES

Part 2. The big picture: how the menu builds itself

The registry is the cleverest part of `core`: no template lists the reports, yet every page shows them. This is the order things happen in:



Three files cooperate: `apps.py` registers at start-up, `reports.py` holds the list, and `context_processors.py` hands it to every template. Add an app, and its `ready()` adds it to the menu: no template changes.

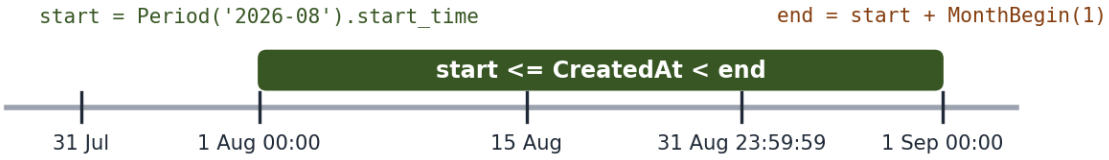
Part 3. Build them, one file at a time

Step 1. periods.month_range: a month as a half-open range

What I was thinking
"Every report filters by month. If each view builds its own date range, one of them will eventually use '<= 31 Aug 23:59:59' and miss the last second, or include 1 Sep."
"pandas already knows months: a Period. Its start_time is the first instant; adding one MonthBegin gives the next month's first instant. That's the half-open range, with no day counting."
"Return both ends as a tuple, so callers write start, end = month_range(run.month)."

```
import pandas as pd


def month_range(month: str) -> tuple[pd.Timestamp, pd.Timestamp]:
    """'2026-08' -> (2026-08-01 00:00, 2026-09-01 00:00): a half-open range."""
    start = pd.Period(month, freq="M").start_time
    return start, start + pd.offsets.MonthBegin(1)
```



Every timestamp in August is inside; 1 Sep 00:00 is not. No 23:59:59 guesswork, no double-counting.

Tool	What it does	Why here
`month: str`	A type hint: 'expects text'	Documents the input; Python doesn't enforce it
`-> tuple[pd.Timestamp, ...]`	A hint for the return value	Readers and VS Code know what comes back
`pd.Period(month, freq='M')`	A whole calendar month as one value	No need to know how many days it has
`.start_time`	The first instant of the period	2026-08-01 00:00:00
`pd.offsets.MonthBegin(1)`	"Move to the next month start"	The exclusive end of the range
`return start, end`	Returns a tuple	Unpacked by the caller

Check yourself
month_range('2026-12') should be (2026-12-01, 2027-01-01): the year rolls over by itself.

Step 2. periods.previous_month: a sensible default

What I was thinking
"The form should open on last month, since reports are usually run for the month that just ended."
"If the function reads today's date inside, I can't test January without waiting for January. Let the caller pass a date, and use today only when nothing is passed."

```
def previous_month(today=None) -> str:
    """The month before today as 'YYYY-MM' (2026-10 in November)."""
    today = pd.Timestamp.today() if today is None else pd.Timestamp(today)
    return (today.to_period("M") - 1).strftime("%Y-%m")
```

Tool	What it does	Why here
`today=None`	An optional parameter	Tests pass a date; the app passes nothing
`a if cond else b`	A one-line if/else expression	Today, or the given date
`pd.Timestamp(x)`	Turns text or a date into a Timestamp	Accepts '2026-01-15' or a datetime
`.to_period('M') - 1`	The month, minus one month	January 2026 becomes December 2025
`.strftime('%Y-%m')`	Formats as text	'2025-12', exactly what the month field expects

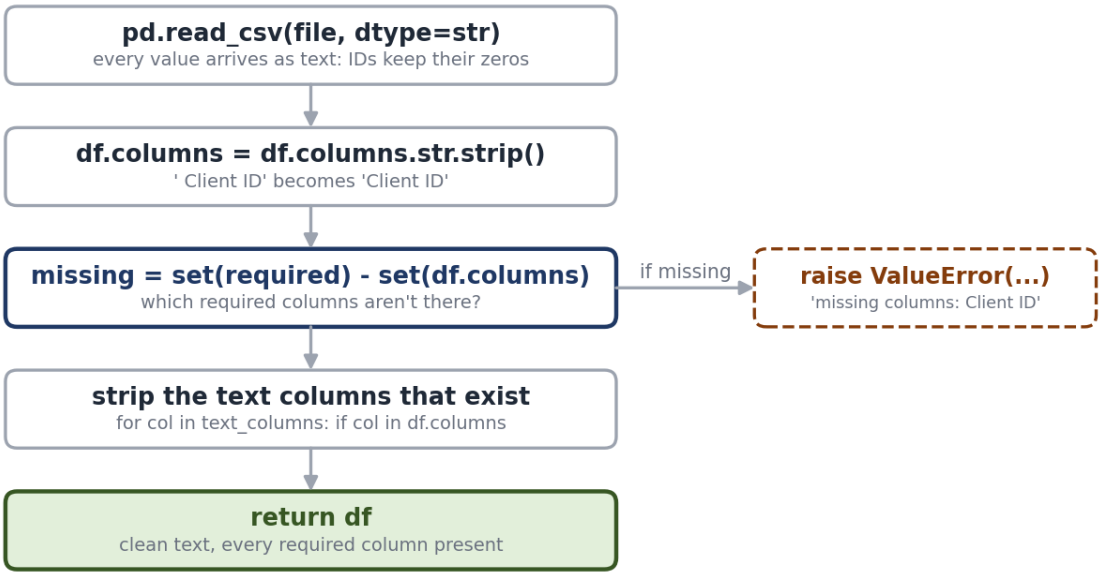
Check yourself
previous_month('2026-01-15') should be '2025-12'.

Step 3. readers.read_csv: one reader with clear errors

What I was thinking
"Every app reads an uploaded CSV, and every upload can be wrong. The user should see 'missing columns: Client ID', not a KeyError three functions later."
"dtype=str for everything: client IDs must stay text, and the app's code decides what becomes a number."
"Exports often have stray spaces in headers, so clean the headers before checking them."
"Some text columns are optional, so only strip the ones that exist."

```
def read_csv(file, required, text_columns=()) -> pd.DataFrame:
    """Read a CSV (a path or an uploaded file), check required columns, strip text columns.

    Raises ValueError with a message a user can act on when columns are missing.
    """
    df = pd.read_csv(file, dtype=str)
    df.columns = df.columns.str.strip()
    missing = set(required) - set(df.columns)
    if missing:
        raise ValueError(f"The file is missing columns: {', '.join(sorted(missing))}")
    for col in text_columns:
        if col in df.columns:
            df[col] = df[col].str.strip()
    return df
```



Tool	What it does	Why here
`pd.read_csv(file, dtype=str)`	Every column read as text	No float IDs, no lost leading zeros
`df.columns.str.strip()`	Removes spaces around every header	' Client ID' still counts as Client ID
`set(a) - set(b)`	Items in a but not in b	The required columns that are missing
`raise ValueError(msg)`	Stops with an error carrying your message	The view shows it on the form
`', '.join(sorted(missing))`	A readable, stable list	Same message order every time
`text_columns=()`	A default of an empty tuple	Safe: a tuple can't be changed by accident

Check yourself
A file whose header is ' Client ID' (leading space) reads fine; a file with no Client ID column raises "The file is missing columns: Client ID".

Why a tuple default and not a list
A default like `text_columns=[]` is created once, when Python reads the def, and shared by every call. If any call ever appended to it, the next call would see that change. A tuple can't be changed, so `()` is always safe.

Step 4. web.download: hand a stored file to the browser

What I was thinking
"Every app has a Download Excel button. The view only needs the stored file and the name the user should see."
"FileResponse streams the file in pieces; as_attachment=True makes the browser save it instead of opening it."

```
from django.http import FileResponse


def download(field_file, filename):
    """Send a stored file to the browser as a download."""
    return FileResponse(field_file.open("rb"), as_attachment=True, filename=filename)
```

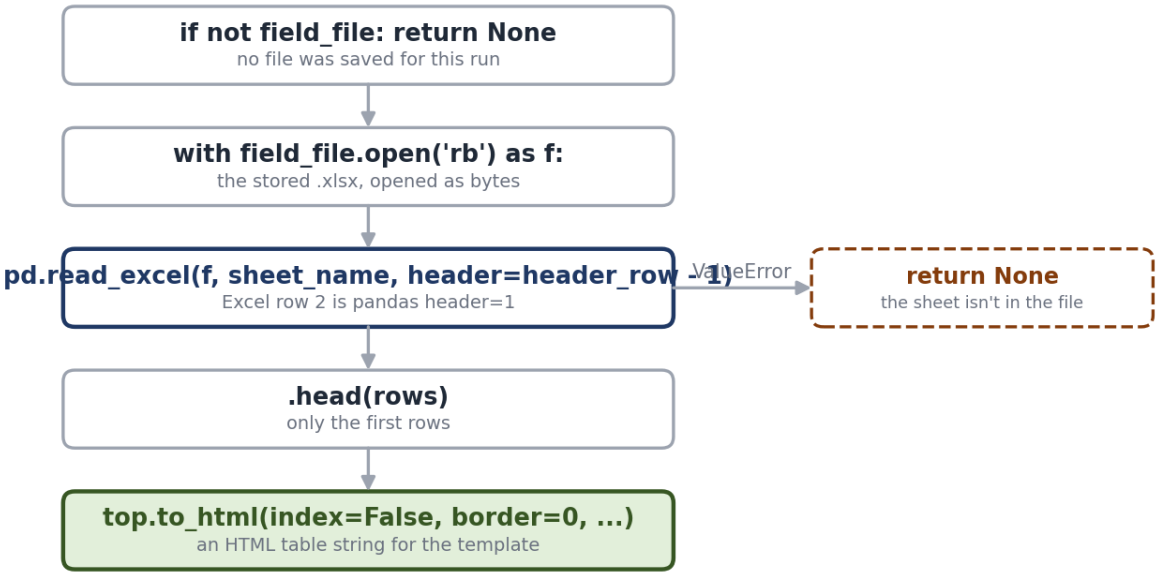
Tool	What it does	Why here
`field_file.open('rb')`	Opens the stored file for reading bytes	Works for any FileField value
`FileResponse(f, ...)`	A response that streams a file	Large files never sit in memory at once
`as_attachment=True`	Sets Content-Disposition: attachment	The browser downloads instead of displaying
`filename=...`	The name the user's browser saves	'Broker_Commission_2026-08.xlsx', not a storage path

Check yourself
Download a run in the browser: the file name should be the report name and month, not the media path.

Step 5. web.sheet_preview: a table for the detail page

What I was thinking
"The detail page should show the first rows of the result, read back from the saved Excel, so the page always matches the file."
"Two harmless failures: no file saved yet, or the sheet doesn't exist (activity has no Manager Summary when the upload had no Manager column). Both should mean 'no preview', not an error page."
"People count Excel rows from 1; pandas counts header rows from 0. Convert in one place."

```
def sheet_preview(field_file, sheet_name, header_row=1, rows=15):
    """The first rows of one sheet of a saved workbook, as an HTML table (None if the file or sheet is missing)."""
    if not field_file:
        return None
    with field_file.open("rb") as f:
        try:
            top = pd.read_excel(f, sheet_name=sheet_name, header=header_row - 1).head(rows)
        except ValueError:                      # pandas raises ValueError for a sheet that isn't there
            return None
    return top.to_html(index=False, border=0, float_format="{:,.2f}".format, na_rep="")
```



Tool	What it does	Why here
`if not field_file`	An empty FileField counts as False	No file yet: no preview
`with ... as f:`	Opens, and always closes, the file	No file handles left open
`try / except ValueError`	Catches one specific error type	Only a missing sheet becomes None; other errors still show
`header=header_row - 1`	Excel row number to pandas index	Banded sheets have headers on Excel row 2
`to_html(index=False, border=0, ...)`	A DataFrame as an HTML table	Ready to drop into a template
`float_format='{:,.2f}'.format`	A function that formats each float	1234.5 becomes 1,234.50

Check yourself
A workbook without a 'Manager Summary' sheet returns None for that name, and HTML for 'Clients'.

Catch only what you expect
A bare `except:` would also hide a corrupt file or a bug in your code. Catching only `ValueError` turns the one expected failure into None and lets every other problem surface.

Step 6. reports.py: the registry

What I was thinking
"The home page and the menu need a list of reports: title, description, where to go. If templates list them by hand, every new app means editing core templates."
"So each app adds itself to one shared list. A dataclass holds the four facts; frozen=True means nobody can change a Report after it's made."
"In tests, Django can run ready() more than once, so register must ignore a key it already has."

```
from dataclasses import dataclass


@dataclass(frozen=True)
class Report:
    key: str           # short id, usually the app label
    title: str         # shown on the home page and in the menu
    description: str
    url_name: str      # where the report starts, e.g. "commission:run_list"


REPORTS: list[Report] = []


def register(report: Report):
    if all(r.key != report.key for r in REPORTS):     # ready() can run more than once in tests
        REPORTS.append(report)
```

Tool	What it does	Why here
`@dataclass`	Writes __init__, __repr__ and __eq__ for you	Report(key=..., title=...) just works
`frozen=True`	Fields can't be changed after creation	A registered report can't be altered by accident
`REPORTS: list[Report] = []`	A module-level list, with a type hint	One shared list while the server runs
`all(... for r in REPORTS)`	True if the condition holds for every item	True when no existing report has this key
`REPORTS.append(report)`	Adds to the list	The report now appears everywhere

Check yourself
Register the same Report twice: REPORTS should contain it once.

Step 7. context_processors.report_apps: data for every template

What I was thinking
"base.html draws the menu, and every view renders a template that extends it. Passing REPORTS from every view would mean editing every view."
"A context processor is a function Django calls for every template render: whatever dict it returns is merged into the template's variables."

```
from .reports import REPORTS


def report_apps(request):
    """Makes the list of report apps available to every template (the menu)."""
    return {"report_apps": REPORTS}


# settings.py, inside TEMPLATES -> OPTIONS -> context_processors:
#     'core.context_processors.report_apps',
```

Tool	What it does	Why here
`def report_apps(request)`	Django passes the current request	Could filter by user later (permissions)
`return {'report_apps': ...}`	Keys become template variables	{% for report in report_apps %} in base.html
`from .reports import REPORTS`	A relative import: same package	core.context_processors imports core.reports
`TEMPLATES[...]['context_processors']`	The list Django calls on every render	Without the entry, the function never runs

Check yourself
Remove the settings line temporarily: the menu should disappear while the home page cards stay, because the home view passes REPORTS itself.

Step 8. apps.py: registering at start-up

What I was thinking
"Registration must happen once, when Django starts, and before any request. That's what AppConfig.ready() is for."
"Import core.reports inside ready(), not at the top of the file: apps.py is imported while Django is still loading apps, and importing other app modules then can fail with AppRegistryNotReady."

```
from django.apps import AppConfig


class CommissionConfig(AppConfig):
    name = "commission"
    verbose_name = "Broker commission"

    def ready(self):
        from core.reports import Report, register
        register(Report(key="commission", title="Broker commission",
                        description="Monthly brokerage per broker from referred clients' P&L, as the MIS Excel.",
                        url_name="commission:run_list"))
```

Tool	What it does	Why here
`class X(AppConfig)`	Configuration for one app	Django finds it when the app is in INSTALLED_APPS
`name = 'commission'`	The app's import path	Must match the folder
`verbose_name`	A readable name	Shown in the admin
`def ready(self)`	Runs once when Django has loaded every app	The right moment to register
`import inside the method`	Delays the import until ready() runs	Avoids AppRegistryNotReady at start-up

Check yourself
Add a third app's Report in its ready(), include its URLs, restart the server: it appears on the home page and in the menu.

Register only after the URLs exist
The menu builds each link with `{% url report.url_name %}`. If an app registers before its URLs are included in `nexora_reports/urls.py`, that lookup fails on every page with NoReverseMatch.

Step 9. core views and URLs: the home page

What I was thinking
"The home page only needs the list. Login first, because these are internal reports."
"app_name = 'core' gives the URL a namespace, so templates write {% url 'core:home' %} and it can't clash with another app's 'home'."

```
# core/views.py
from django.contrib.auth.decorators import login_required
from django.shortcuts import render

from .reports import REPORTS


@login_required
def home(request):
    return render(request, "core/home.html", {"reports": REPORTS})


# core/urls.py
from django.urls import path

from . import views

app_name = "core"
urlpatterns = [path("", views.home, name="home")]
```

Tool	What it does	Why here
`@login_required`	Redirects anonymous users to LOGIN_URL	Reports stay private
`render(request, template, context)`	Fills a template with data	Returns the HTML response
`app_name = 'core'`	The URL namespace	{% url 'core:home' %}
`path('', view, name='home')`	The empty path inside the include	The project mounts core at '/'

Check yourself
Log out and open '/': you should land on the login page, then return to the home page after logging in.

Part 4. Mistakes these helpers prevent

If you wrote...	What would happen	What core does instead
Each view building its own month range	One report uses <= 23:59:59 and misses rows	month_range() in one place
date.today() inside previous_month	January can't be tested until January	today=None, pass a date in tests
pd.read_csv(file) without dtype	IDs as floats, leading zeros lost	dtype=str, convert numbers deliberately
df['Client ID'] with no check	KeyError deep inside the calculation	read_csv lists missing columns clearly
text_columns=[] as a default	A shared, changeable default list	text_columns=()
except: in sheet_preview	Real errors silently become 'no preview'	except ValueError only
A menu typed into base.html	core edited for every new app	The registry + context processor
from core.reports import ... at the top of apps.py	AppRegistryNotReady at start-up	Import inside ready()
register() without the key check	Reports listed twice in tests	all(r.key != report.key ...)

Part 5. Checkpoints

Rename the real files (for example `periods_reference.py`), rewrite them, and run these tests. They pass against the project's files.

```
from io import BytesIO

import pandas as pd
from django.test import SimpleTestCase

from core import reports
from core.excel import SheetSpec, workbook_bytes
from core.periods import month_range, previous_month
from core.readers import read_csv
from core.web import sheet_preview


class FakeFile:                                  # stands in for a FieldFile: truthy, with open()
    def __init__(self, data): self.data = data
    def __bool__(self): return True
    def open(self, mode): return BytesIO(self.data)


class HelperCheckpoints(SimpleTestCase):
    def test_month_range_rolls_over_the_year(self):
        self.assertEqual(month_range("2026-12"), (pd.Timestamp("2026-12-01"), pd.Timestamp("2027-01-01")))

    def test_previous_month_in_january(self):
        self.assertEqual(previous_month("2026-01-15"), "2025-12")

    def test_read_csv_strips_and_checks(self):
        df = read_csv(BytesIO(b" Client ID ,Name\n C1 ,x\n"), required=["Client ID"], text_columns=["Client ID"])
        self.assertEqual(df.loc[0, "Client ID"], "C1")
        with self.assertRaisesMessage(ValueError, "missing columns: Client ID"):
            read_csv(BytesIO(b"ID\n1\n"), required=["Client ID"])

    def test_sheet_preview(self):
        data = workbook_bytes([SheetSpec("Clients", pd.DataFrame({"A": [1.5]}))])
        self.assertIn("<th>A</th>", sheet_preview(FakeFile(data), "Clients"))
        self.assertIsNone(sheet_preview(FakeFile(data), "Manager Summary"))

    def test_register_ignores_a_repeated_key(self):
        saved = list(reports.REPORTS)
        try:
            r = reports.Report("tmp", "Tmp", "x", "core:home")
            reports.register(r); reports.register(r)
            self.assertEqual(sum(x.key == "tmp" for x in reports.REPORTS), 1)
        finally:
            reports.REPORTS[:] = saved           # leave the shared list as it was
```

`reports.REPORTS[:] = saved` replaces the list's contents in place. Writing `reports.REPORTS = saved` would create a new list that the context processor never sees, because it imported the original.

Explain it back

Answer each in one or two sentences, without looking. The answers follow.

1. Why does month_range return the first instant of the next month instead of the last second of this one?
2. Why does previous_month take a today parameter?
3. Why is every column read with dtype=str?
4. What does set(required) - set(df.columns) give you?
5. Why does sheet_preview catch ValueError only?
6. What does frozen=True add to the Report dataclass?
7. When does a context processor run, and what does it return?
8. Why is core.reports imported inside ready()?

Answers

1. A half-open range (>= start, < end) catches every timestamp in the month, including fractions of a second, and never overlaps the next month.
2. So tests can pass a fixed date. Reading today inside would make January impossible to test in November.
3. So IDs stay exactly as typed. The services decide which columns become numbers.
4. The required columns that aren't in the file, ready for the error message.
5. A missing sheet is expected and should mean 'no preview'; anything else is a real problem that should show.
6. A Report can't be modified after it's created, so the registry can't be changed by accident.
7. On every template render; its dict is merged into the template's variables.
8. apps.py is imported while Django is still loading apps; importing other app modules then can raise AppRegistryNotReady.

Appendix. The files, with line numbers

core/periods.py

```
 1  """Date helpers shared by every report. Plain pandas: no Django imports."""
 2  import pandas as pd
 3
 4
 5  def month_range(month: str) -> tuple[pd.Timestamp, pd.Timestamp]:
 6      """'2026-08' -> (2026-08-01 00:00, 2026-09-01 00:00): a half-open range."""
 7      start = pd.Period(month, freq="M").start_time
 8      return start, start + pd.offsets.MonthBegin(1)
 9
10
11  def previous_month(today=None) -> str:
12      """The month before today as 'YYYY-MM' (2026-10 in November)."""
13      today = pd.Timestamp.today() if today is None else pd.Timestamp(today)
14      return (today.to_period("M") - 1).strftime("%Y-%m")
```

Lines	Block	Purpose
5-8	`month_range()`	Half-open month
11-14	`previous_month()`	Default month for forms

core/readers.py

```
 1  """Reading uploaded files with clear errors. Plain pandas: no Django imports."""
 2  import pandas as pd
 3
 4
 5  def read_csv(file, required, text_columns=()) -> pd.DataFrame:
 6      """Read a CSV (a path or an uploaded file), check required columns, strip text columns.
 7
 8      Raises ValueError with a message a user can act on when columns are missing.
 9      """
10      df = pd.read_csv(file, dtype=str)
11      df.columns = df.columns.str.strip()
12      missing = set(required) - set(df.columns)
13      if missing:
14          raise ValueError(f"The file is missing columns: {', '.join(sorted(missing))}")
15      for col in text_columns:
16          if col in df.columns:
17              df[col] = df[col].str.strip()
18      return df
```

Lines	Block	Purpose
5-18	`read_csv()`	Read, check, clean

core/web.py

```
 1  """Small view helpers every report app reuses."""
 2  import pandas as pd
 3  from django.http import FileResponse
 4
 5
 6  def download(field_file, filename):
 7      """Send a stored file to the browser as a download."""
 8      return FileResponse(field_file.open("rb"), as_attachment=True, filename=filename)
 9
10
11  def sheet_preview(field_file, sheet_name, header_row=1, rows=15):
12      """The first rows of one sheet of a saved workbook, as an HTML table (None if the file or sheet is missing)."""
13      if not field_file:
14          return None
15      with field_file.open("rb") as f:
16          try:
17              top = pd.read_excel(f, sheet_name=sheet_name, header=header_row - 1).head(rows)
18          except ValueError:                      # pandas raises ValueError for a sheet that isn't there
19              return None
20      return top.to_html(index=False, border=0, float_format="{:,.2f}".format, na_rep="")
```

Lines	Block	Purpose
6-8	`download()`	File to browser
11-20	`sheet_preview()`	HTML preview or None

core/reports.py

```
 1  """A registry of report apps: each app adds itself in its AppConfig.ready(), the home page lists them."""
 2  from dataclasses import dataclass
 3
 4
 5  @dataclass(frozen=True)
 6  class Report:
 7      key: str           # short id, usually the app label
 8      title: str         # shown on the home page and in the menu
 9      description: str
10      url_name: str      # where the report starts, e.g. "commission:run_list"
11
12
13  REPORTS: list[Report] = []
14
15
16  def register(report: Report):
17      if all(r.key != report.key for r in REPORTS):     # ready() can run more than once in tests
18          REPORTS.append(report)
```

Lines	Block	Purpose
5-10	`Report`	Four facts per report
13	`REPORTS`	The shared list
16-18	`register()`	Add once per key

core/context_processors.py

```
 1  from .reports import REPORTS
 2
 3
 4  def report_apps(request):
 5      """Makes the list of report apps available to every template (the menu)."""
 6      return {"report_apps": REPORTS}
```

commission/apps.py

```
 1  from django.apps import AppConfig
 2
 3
 4  class CommissionConfig(AppConfig):
 5      name = "commission"
 6      verbose_name = "Broker commission"
 7
 8      def ready(self):
 9          from core.reports import Report, register
10          register(Report(key="commission", title="Broker commission",
11                          description="Monthly brokerage per broker from referred clients' P&L, as the MIS Excel.",
12                          url_name="commission:run_list"))
```

Lines	Block	Purpose
8-12	`ready()`	Registers the app at start-up

core/views.py

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render

from .reports import REPORTS


@login_required
def home(request):
    return render(request, "core/home.html", {"reports": REPORTS})
```

core/urls.py

```
from django.urls import path

from . import views

app_name = "core"
urlpatterns = [path("", views.home, name="home")]
```